

Render pipelines

Entendiéndolos con Unity

Marco Antonio Gómez Martín
Iluminación y Materiales
Grado en Desarrollo de Videojuegos
2025/2026



Facultad
de
Informática



UNIVERSIDAD COMPLUTENSE
MADRID

Frame buffer

- Función `display()` de la ventana con doble buffer: **Front** y **Back**:

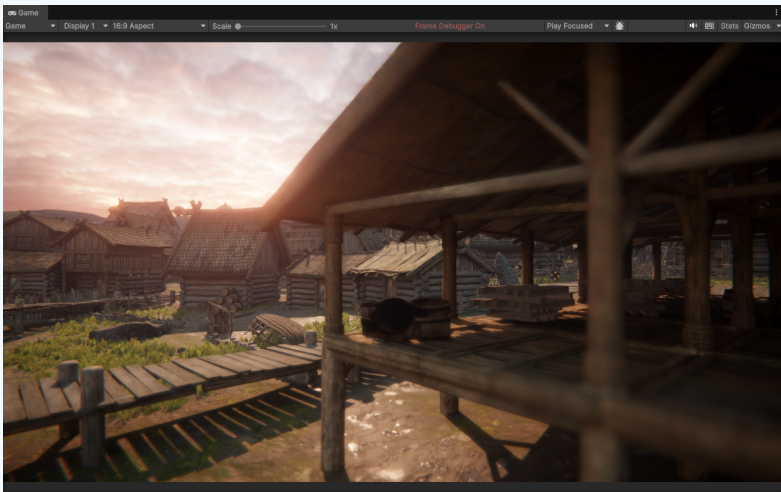
```
// El BackColorBuffer queda del color fijado
// con glClearColor()
// El DepthBuffer queda a 1 (máxima distancia)
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

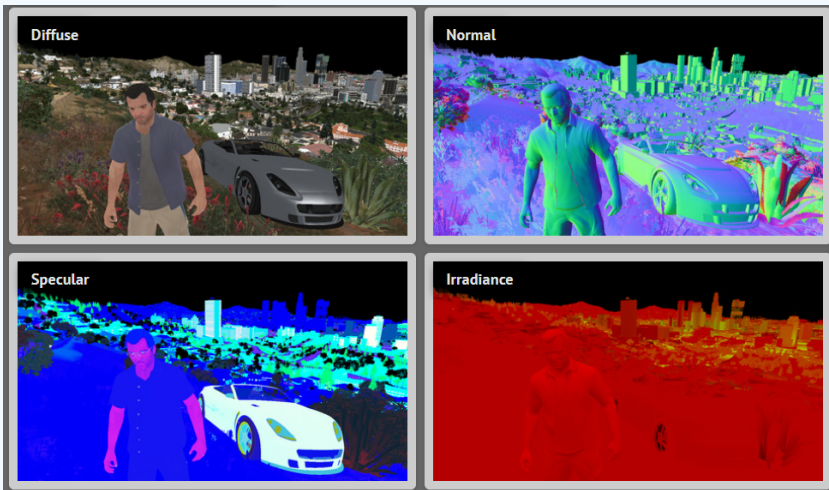
// dibuja los objetos en el BackColorBuffer
scene.render();

// intercambia los buffers (Back/Front)
glfwSwapBuffers(mWindow);
```

Con ese esquema, hay cosas difíciles/imposibles de conseguir:

- Transparencias
- Sombras
- Espejos
- Mini mapas
- Efectos de postprocesado
- ...





Render pipelines

Render pipeline

A render pipeline performs a series of operations that take the contents of a scene, and displays them on a screen.

Render paths

A rendering path is the series of operations that draws and lights GameObjects the camera sees. Different rendering paths have different capabilities and performance characteristics.

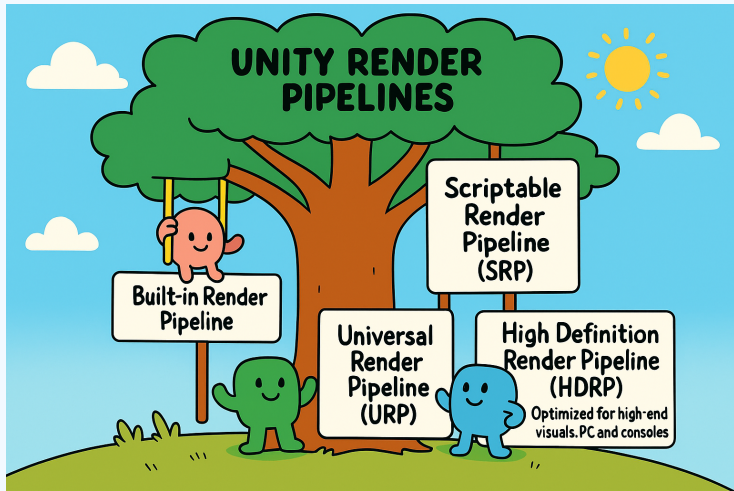
Unity render pipelines

- Built-In Render Pipeline
- Universal Render Pipeline (URP)
- High Definition Render Pipeline (HDRP)

Unity render pipelines

- Built-In Render Pipeline
- Scriptable Render pipelines (SRP)
 - Universal Render Pipeline (URP)
 - High Definition Render Pipeline (HDRP)

Unity render pipelines



<https://www.4rknova.com/blog/2025/08/04/unity-render-pipelines>

Unity render pipelines

SRP hace accesible el proceso de renderizado a C#:

- Control total sobre qué objetos se dibujan.
- Posibilidad de crear render pipelines desde cero.
- Posibilidad de optimizar el pipeline para usos y hardware concretos.
- Tecnología más moderna hacia la que están dirigidos avances posteriores como *Shader graph* o los *Render graphs*.

Unity render pipelines y shaders

- Los shaders son específicos de cada render pipeline
- API/funciones diferentes proporcionadas por el motor
- Etapas de renderizado distintas, con pasadas del shader específicas para etapas concretas
- El mismo `.shader` puede servir para todos los render pipelines haciendo uso de macros

Unity render pipelines y shaders

```
Shader "Custom/Ejemplo" {  
    Properties { ... }  
  
    // URP SubShader  
    #ifdef UNITY_PIPELINE_UNIVERSAL  
    SubShader { ... }  
    #endif  
  
    // HDRP SubShader  
    #ifdef UNITY_PIPELINE_HD  
    SubShader { ... }  
    #endif  
  
    // Built-in RP fallback SubShader  
    #if !defined(UNITY_PIPELINE_HD) && !defined(UNITY_PIPELINE_UNIVERSAL)  
    SubShader { ... }  
    #endif  
}
```

Folletín - Built-in Render Pipeline

Pros

Contras

Maduro y estable

Compatible con gran cantidad de assets y shaders

Soporte para muchas plataformas

Fácilmente extensible con shaders y efectos

No implementa últimas tendencias en optimizaciones

Difícil de mantener y escalar en sistemas modernos

No tiene soporte para tecnologías más modernas (shader graph, ...)

Folletín - Built-in Render Pipeline

Características

- Forward y deferred
- Shaders basados en el sistema "legacy". Con *Surface shaders*.

Cuándo usarlo

- A estas alturas... ¿nunca? :-)
- Proyectos que confían en assets y shaders antiguos.
- Necesidad de compatibilidad con el máximo número de plataformas.
- Prototipos rápidos sin gráficos sofisticados.

Folletín - Universal Render Pipeline

Pros

Contras

Multilataforma y escalable

Mejor rendimiento que el built-in RP
en la mayoría de los dispositivos

Soporte para Shader graph

Soporte cómodo para postprocesa-
do o volúmenes

Capacidades gráficas limitadas si las
comparamos con HDRP

No compatible con assets y shaders
del built-in RP.

Folletín - Built-in Render Pipeline

Características

- Muy configurable gracias a ser un SRP.
- Soporta Shader Graph.
- Optimizado para funcionar bien en hardware limitados.

Cuándo usarlo

- Móviles, VR, AR... ("low to mid-range hardware").
- Cuando se persigue un buen balance entre rendimiento y calidad.

Folletín - High Definition Render Pipeline

Pros	Contras
Incorpora las últimas tendencias en rendering	No adecuado para hardware con capacidades limitadas
Usa técnicas de renderizado y post-procesado avanzadas	Más complicado de configurar y optimizar
Ideal para proyectos de arquitectura, juegos AAA y cine	No compatible con assets y shaders del built-in RP ni URP.

Folletín - Built-in Render Pipeline

Características

- Deferred por defecto.
- Soporta ray tracing, iluminación volumétrica, ...
- Necesita GPUs modernas de alto rendimiento

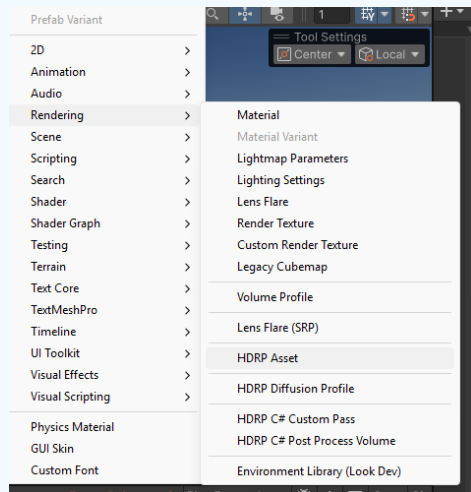
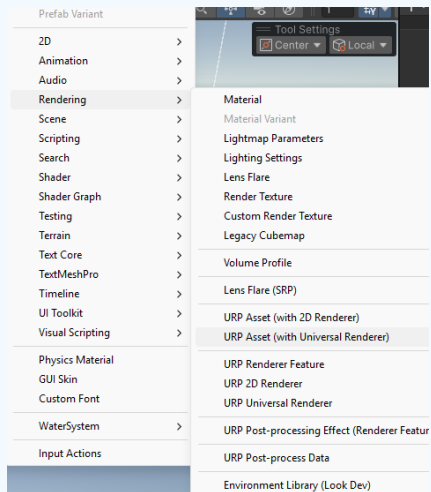
Cuándo usarlo

- Proyectos para PCs de gama alta y consolas de última generación con uso intensivo de gráficos
- Proyectos de arquitectura y cine
- En general, cuando el proyecto exige una alta calidad en el resultado final

SRP a vista de pájaro

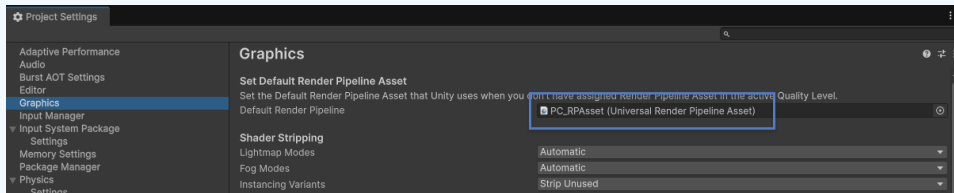
- Clase `UnityEngine.Rendering.RenderPipeline`. Implementa el método `Render`.
- Clase `RenderPipelineAsset`. Tiene el método `CreatePipeline` que instancia el `RenderPipeline` que se utiliza.
 - Es un *asset* del proyecto en el que se añaden propiedades que pueden cambiarse en el editor.
 - En un mismo proyecto puede haber varios.
 - Se puede utilizar uno u otro dependiendo de la configuración.
 - Unity utilizará el seleccionado para todas las cámaras del proyecto, incluyendo la pestaña *Scene* o las previsualizaciones en los *Inspectors*.
- Clase `ScriptableRenderContext`: recibido por el método `Render`, contiene el API para acceder al código de bajo nivel de Unity.

SRP a vista de pájaro - Creación del asset

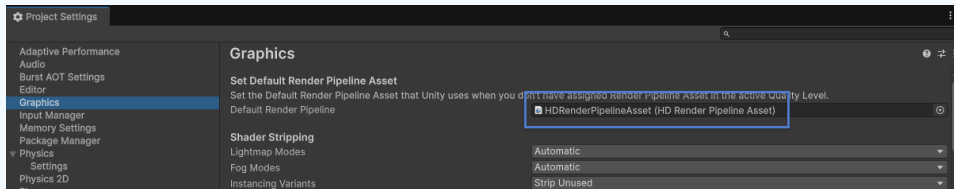


SRP a vista de pájaro

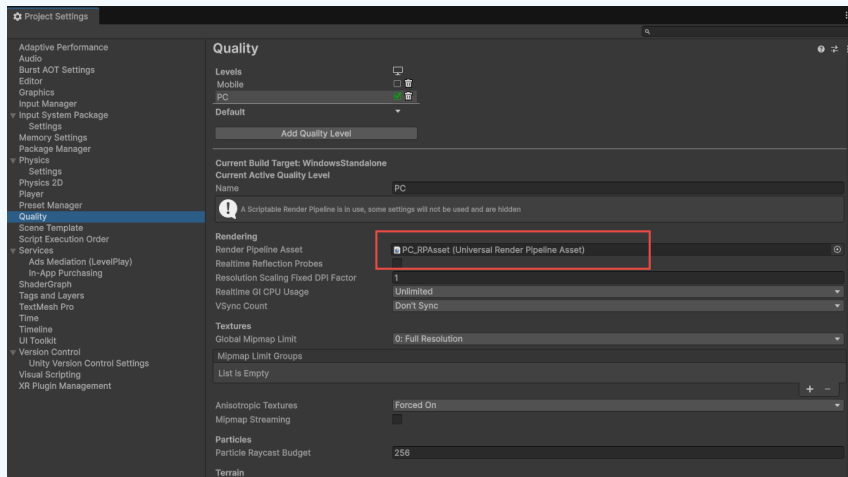
URP



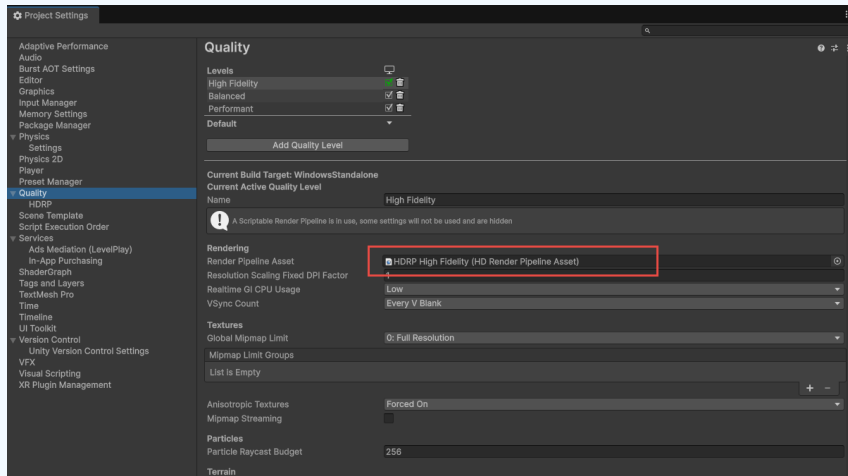
HDRP



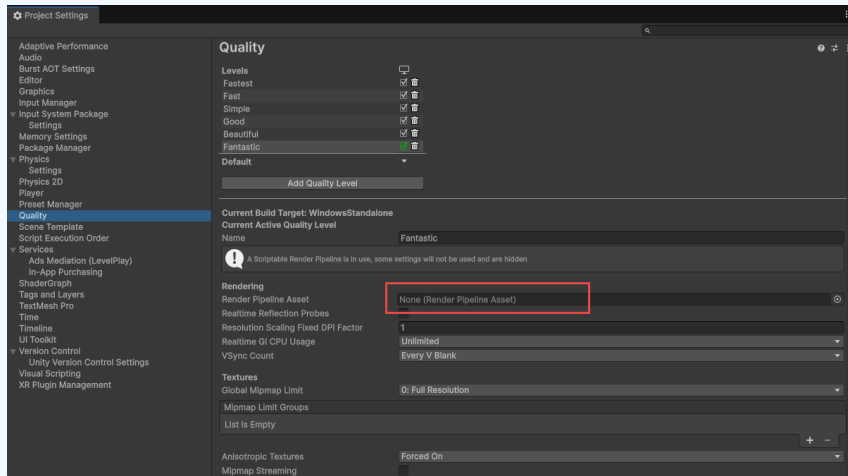
SRP a vista de pájaro - Quality settings



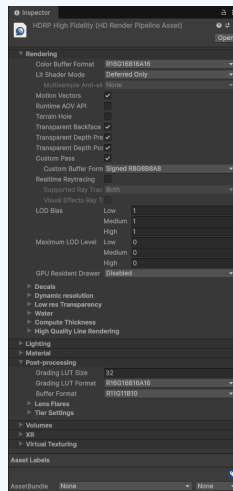
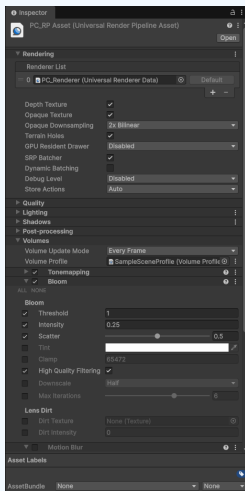
SRP a vista de pájaro - Quality settings



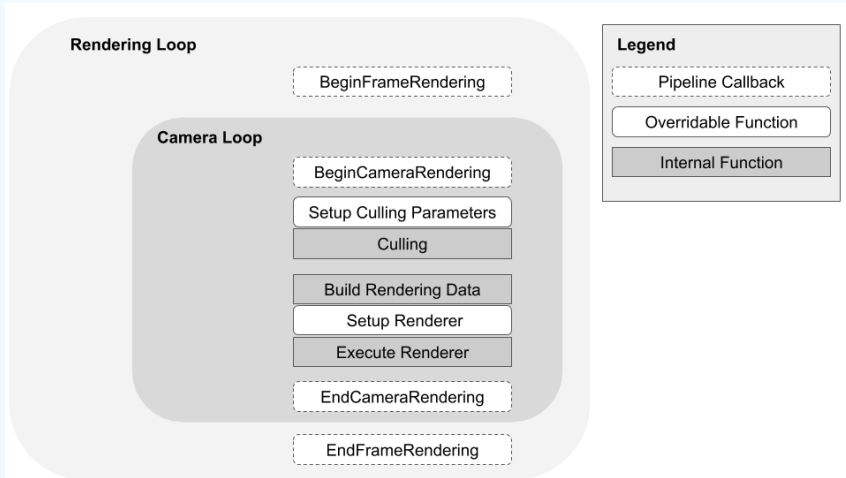
SRP a vista de pájaro - Quality settings



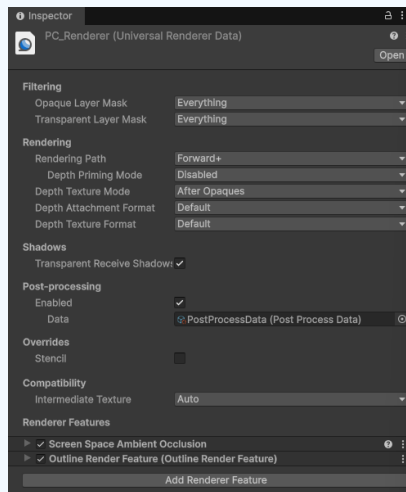
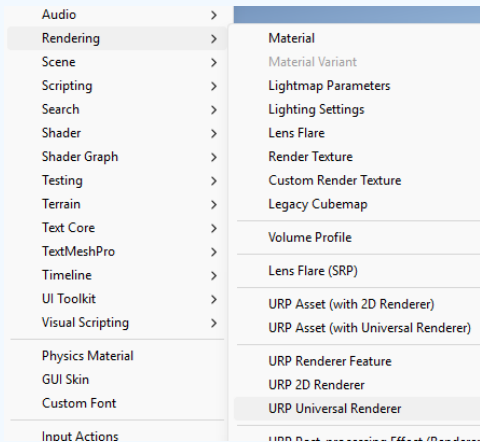
SRP a vista de pájaro - Propiedades del Render Pipeline Asset



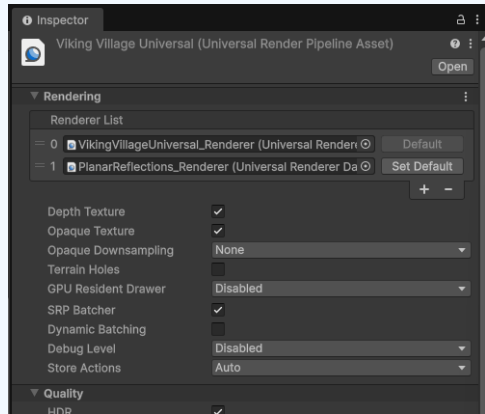
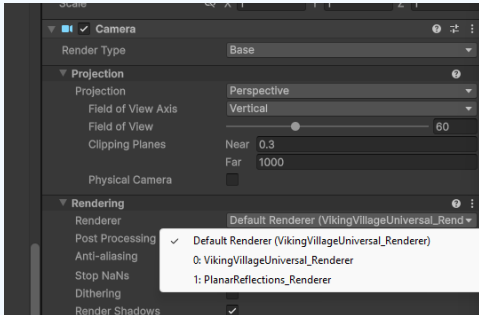
Rendering loop



URP - Universal Renderer Data



URP - Universal Renderer Data

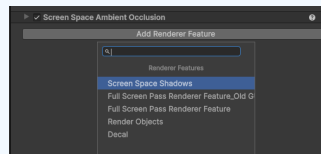
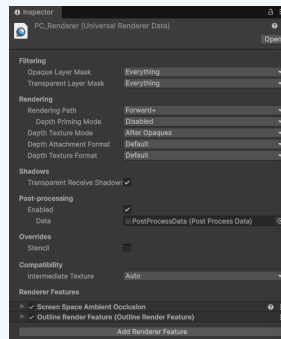


iDEMO!

URP - Render passes

URL permite añadir nuevos pases:

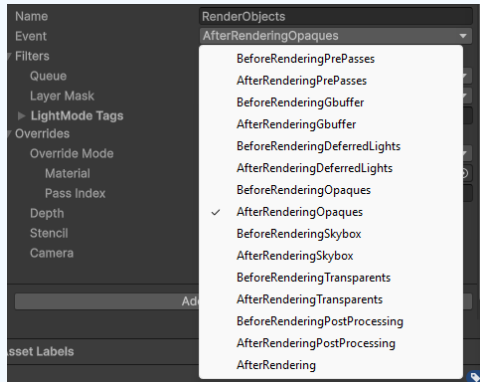
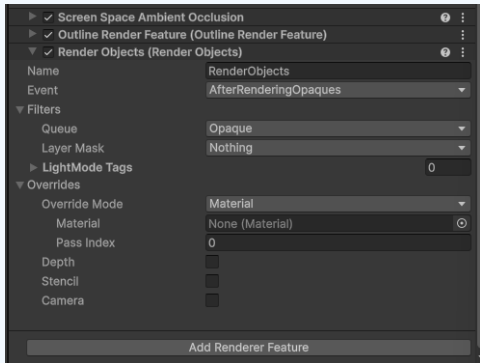
- Añadiendo *Render features* al *Renderer*
 - Utilizando las ya construidas que pueden configurarse y utilizarse
 - Creando nuevas (C#).
- Añadiendo pasadas (desde componentes, por ejemplo) utilizando API específico (`RenderPipelineManager`).



URP - Render objects feature

- Pasada nueva para renderizar objetos/geometría.
- Permite filtrar (con máscaras) que objetos renderizar.
- Permite configurar en qué momento del montaje del frame tiene lugar.
- Permite sobrescribir características de los objetos:
 - Materiales
 - Comportamiento del z-buffer
 - Comportamiento del stencil
 - ...

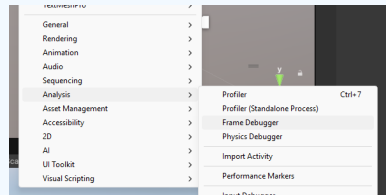
URP - Universal Renderer Data



iDEMO!

Frame Debugger

- En Windows - Analysis - Frame Debugger.
- En la ventana que se abre, pulsar "Enable".
- Permite ver los distintos pasos en la construcción del frame.
- Muestra los resultados intermedios.
- Muestra información sobre los batches, geometría utilizada, materiales, shaders, ...



Frame debugger

Frame Debugger

Disable Editor

RT 0 Channels AR R G B A Levels 0 1

124

Event #124 Clear (depth+stencil)

Output



Details

RenderTarget: GameView_RT

Size: 699x393

Format: R32G8B8A8_SRGB

Color Actions: Clear / DontCare

Depth Actions: -

Clear Color: -

Clear Depth: 1.000

Clear Stencil: (00000000) 0

Meshes

Keywords

Textures

Inst

Floats

Vectors

Matrices

Buffers

Constant Buffers

UniversalRenderPipeline.RenderSingleCameraInternal: Planar Reflections 3

UniversalRenderPipeline.RenderSingleCameraInternal: Camera.oop 121

ExecuteRenderGraph 121

Clear (depth+stencil)

(RP 1:0) Draw Main Light Shadowmap 25

Clear (color)

(RP 2:0) Blt Color LUT 1

Clear (color+depth+stencil)

(RP 3:0) DrawDepthNormalPrepass 18

Clear (color)

SSAO 5

Clear (depth+stencil)

(RP 4:0) DrawOpaqueObjects 25

(RP 4:0) DrawSkybox 1

Resolve Color

Clear (color)

(RP 5:0) CopyColor 1

(RP 6:0) DrawTransparentObjects 2

Resolve Color

Clear (color+depth+stencil)

(RP 7:0) RO_SMAAEdgeDetection 1

Clear (color)

(RP 8:0) RO_SMAABlendWeight 1

Clear (color)

(RP 9:0) RO_SMAANeighborhoodBlend 1

Clear (color)

Clear (color)

Clear (color)

Clear (color)

Depth of Field - Bokeh 5

Clear (color)

(RP 10:0) PaniniProjection 1

Bloom 16

(RP 11:0) RO_UltraPost 1

(RP 11:1) DrawScreenSpaceUI 1

UI.RenderOverlays 1

Clear (depth+stencil)

Frame debugger

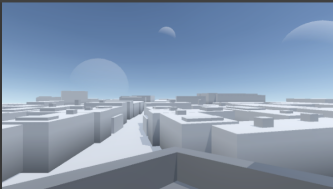
Frame Debugger

Disable Editor

RT 0 Channels AR R G B A Levels 0 55 1

Event #55 Draw Procedural

Output



Details

RenderTarget	GameView RT
Size	699x393
Format	RGB888A8_SRGB
Color Actions	Clear / DontCare
Depth Actions	-
Memoryless	No
Foveated Rendering	Disabled
ColorMask	RGBA
Blend Color	One / Zero
Blend Alpha	One / Zero
BlendOp Color	Add
BlendOp Alpha	Add
DrawInstanced Calls	-
Instances	-
Draw Calls	1
Vertices	3
Indices	8
Batch cause	Unknown reason.
Mesh	-
LightMode	-
Pass	UberPost (#)
Used Shader	Hidden/Universal Render Pipeline/UberPost
Original Shader	Hidden/Universal Render Pipeline/UberPost

Meshes

Frame debugger

The screenshot displays the Unity Frame Debugger interface. The left sidebar shows a tree of rendering events, with 'Event #11 SRP Batch' selected. The main window is divided into three sections: 'Output', 'Details', and 'Meshes'.

Output: A small 3D preview window showing a bright green cube on a dark blue background.

Details: A table of rendering parameters for the selected event.

Property	Value	Property	Value
RenderTarget	CameraTargetAttachment_768x432_B18011R11_UFloatPack32_Tex2D	Stencil	True
Size	768x432	ZTest	LessEqual
Format	B18011R11_UFloatPack32	Write	On
Color Actions	Clear / Store	Cull	Back
Depth Actions	-	Conservative	False
Memoryless	No	Offset	0.0
Fastest Rendering	Disabled		
ColorMask	RGBA	Stencil Ref	-
Blend Color	One / Zero	Stencil ReadMask	-
Blend Alpha	One / Zero	Stencil WriteMask	-
BlendOp Color	Add	Stencil Comp	-
BlendOp Alpha	Add	Stencil Pass	-
DrawInstanced Calls	-	Stencil Fail	-
Instances	-	Stencil ZFail	-
Draw Calls	1		
Vertices	24		
Indices	36		

Batch cause: SRP: First call from ScriptableRenderLoopJob

Meshes: A list of meshes rendered in this batch, with 'Cube' selected.

Mesh	LightMode	Pass	Used Shader	Original Shader
Cube	-	Dibujado trasladado (1)	Unity/D3D/Deferred/DeferredLit	Unity/D3D/Deferred/DeferredLit

Frame debugger

The screenshot displays the Unity Frame Debugger interface. The left sidebar shows a hierarchy of rendering events, with 'Event #12 SRP Batch' selected. The main window is divided into three sections: a top view of the scene showing two cubes (one red, one green) on a dark blue background, a middle 'Details' section, and a bottom 'Meshes' section.

Details

RenderTarget	.CameraTargetAttachment_768x432_B18011R11_UFloatPack32_Tex2D		
Size	768x432	ZClip	True
Format	B18011R11_UFloatPack32	ZTest	LessEqual
Color Actions	Clear / Store	Write	On
Depth Actions	-	Cull	Back
Memoryless	No	Conservative	False
Foveated Rendering	Disabled	Offset	0.0
ColorMask	RGBA	Stencil	-
Blend Color	One / Zero	Stencil Ref	-
Blend Alpha	One / Zero	Stencil ReadMask	-
BlendOp Color	Add	Stencil WriteMask	-
BlendOp Alpha	Add	Stencil Comp	-
DrawInstanced Calls	-	Stencil Pass	-
Instances	-	Stencil Fail	-
Draw Calls	1	Stencil ZFail	-
Vertices	24		
Indices	36		

Batch cause
SRP: Node use multi-pass shader

Meshes

Meshes	Cube
LightMode	UniversalForward
Pass	Dibujado estandar (0)
Used Shader	UniversalForwardPasadas
Original Shader	UniversalForwardPasadas

iDEMO!

Render pipelines

Entendiéndolos con Unity

¿Preguntas?



Facultad
de
Informática

